

Swift Basics

Swift is general purpose, object and protocol oriented programming language which is developed by Apple Inc. for iPad, iPhone, macOS, WatchOS, tvOS and etc. to build Apple products.

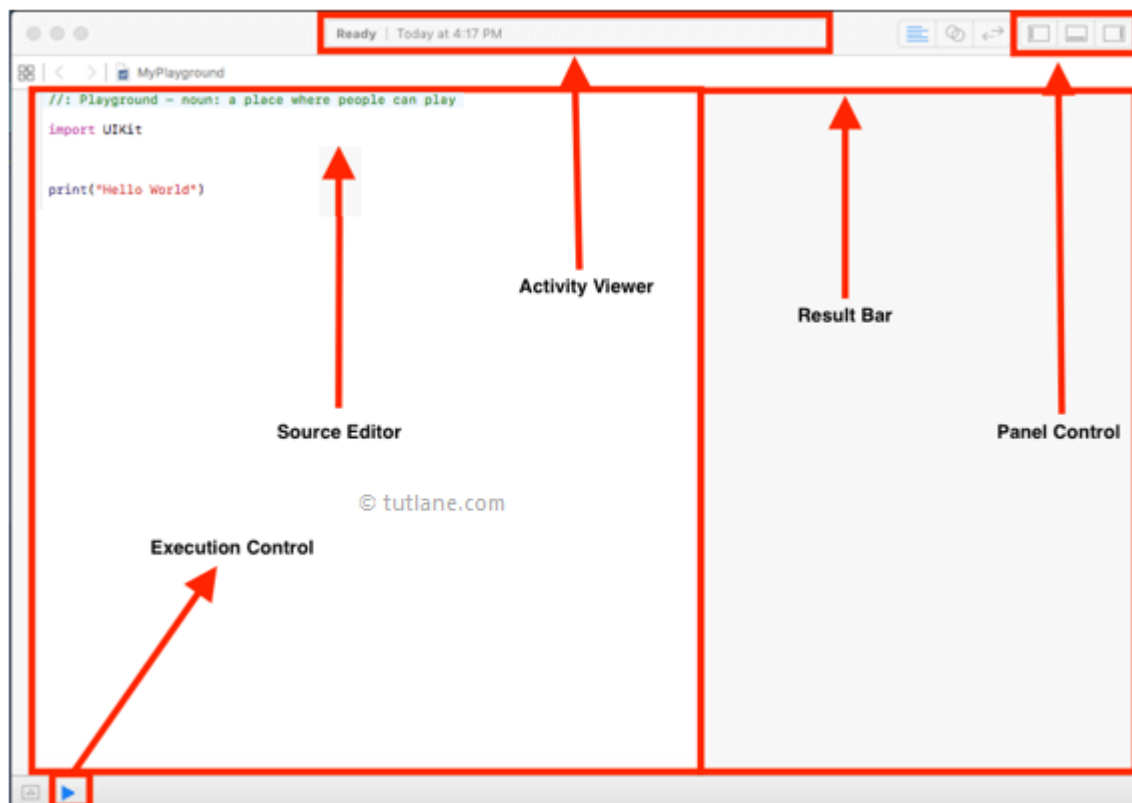
The swift language is designed to work with Apple's Cocoa and Cocoa Touch frameworks and it supports the fundamental concepts of Objective C like multithreading, optional chaining, late binding and some more features. In Swift programming language Apple added new features like automatic memory management, protocols and introduced the protocol oriented programming.

Features of Swift

- When compared with other programming languages, the syntax of Swift is very easy and it allow users to write code just like matching with PHP and etc.
- No need to give the semicolon at the end of statement which help the programmers to concentrate more on the code writing not in semicolons.
- In Objective C, we manually allocate or release the memory but Swift Programming Language Automatic handle the Memory Management.
- Real Time Output Swift Programming Language include the playground feature which will help you to write the code and don't need to run the code separately, automatically it will compile the code and shows the result.
- Apple announced that Swift is the first programming language which is similar with any scripting language.
- In Swift, Structure supports the extensions, protocols and methods.

Swift is a **type-safe** programming language which means it will help you to identify what type of values required to work with the code. Suppose if we are passing "**int**" value but our code required "**string**" in that case type safety will prevent us to send invalid types.

Generally, the swift programming playground editor will contain two portions like as shown below. The left portion of editor is to write the code and right side portion is used to see the result of coding.



If you observe above playground editor by default it contains small piece of code like as shown below.

```
//: playground - noun: a place where people can play
import UIKit
print("Hello World")
```

When we run above program we will get result like as shown below

```
Hello World
```

If you observe above sample code it's following some basic structure like first **comments**, **import** class then required keywords for coding. Now we will see the basic building blocks of Swift programming language.

Comments in Swift

The very first line in playground project which is highlighted with green text is known as **Comments**. In all programming languages, the concept of comments remains same. The Comments are those line which are not compiled by compiler these lines are basically help for developers and programmers to read and understand the code in better way. The Comments are the two types the first type is **Single Line** Comment and the second line is **Multi-Line** Comment.

The declaration of comments like as shown below

Single-line comment

```
//: Playground - noun: a place where people can play
```

Multi-line comments

```
/* Playground - noun: a  
place where  
people can play */
```

Swift Variables

In swift variables are used to store value in memory and we can manipulate or change the value of variables by applying different operations based on our requirements.

We use keyword called “**var**” to define variables in swift. Following is the syntax of defining variable in swift programming language.

```
var<VariableName> = <Value>
```

Following is the simple example of declaring variable in swift and assigning value to it.

```
var str = "IOS Programming"  
print(str)
```

When we execute above code we will get result like as shown following.

```
IOS Programming
```

Swift Constants

Constant means it refers fixed value it means when we declare variable as constant then it's not possible for us to change or override it throughout the program. Generally we declare constant variables with keywords called “**let**”.

Following is the syntax of declaring constant variable.

```
let <ConstantName> = <Value>
```

Following is the simple example of defining constant variable in swift programming language.

```
let str = "IOS Programming"  
print(str)
```

The above statements will return output like as shown following.

```
IOS Programming
```

These are the some of basic syntaxes which we used frequently in swift programming language.

Swift Basic Data Types

Following are the basic and most frequently used data types available in swift programming language.

Data Type	Description	Example
Integer	Int or UInt stand for Integer and used for numbers like 1, 2, 3, 4, 5. We can use Int32, Int64 to define 32 or 64-bit signed integer, whereas UInt32 or UInt64 to define 32 or 64-bit unsigned integer variables. For example, 345566 and -345566.	100
Float	Float stand for floating value and it is used to represent numbers with fraction value like 245.344 and etc. It is used to hold the numbers with larger or smaller decimal points like 3.14, and -455.3344.	3.14 -455.3344
Double	Double stands for Double means if we want to allocate the large number with fraction value then we use Double as a float data-type. We can use it as a 64-bit floating-point number and Double is more precise than Float. For example, 345.344544, and -4554.3455543.	345.344544 -4554.3455543
Bool	Bool represent only two values either TRUE or FALSE. We can use Bool to check whether certain condition met or not.	let i = 10 if i > 20 { // your code }
String	String is a combination of Characters. We can define string data type by adding double quotes to our text like "Hello World".	"" "welcome to iOS Programming"
Character	It represent a single alphabet character like "H", "e", "L", "l", "o".	"H"
Optional	It represent a variable can hold value or no value.	

Swift Variable Concatenation & Interpolation

In swift we can combine two variables into single variable for that we have two ways, one is we can concatenate it and second is to interpolate variable with parenthesis.

Following is the simple example of concatenating two variables into single variable using swift.

```
var var1 = "Welcome"
var var2 = "IOS Programming"

var result = var1 + var2
print(result)
```

The swift programming playground will return the result like as shown below.

```
Welcome IOS Programming
```

Now we will see how to use **interpolation** to combine two variables with examples. Following is simple example of using interpolation to combine two variables in swift.

```
var var1 = "Welcome"  
var var2 = "IOS Programming"  
print("\(var1)/(var2)")
```

The above swift program will return result like as shown below.

```
Welcome IOS Programming
```

Swift Strings Interpolation

In swift string interpolation means creating new string value from a mix of numbers, constants, variables, literals and expressions by including their values inside a string literal.

Every time when we insert into string literal, it is wrapped into pair of parentheses and prefixed by backslash (\).

Following is the simple example of implementing string interpolation in swift programming language.

```
let name = "Bharat Patel"  
let strinter = "Hello My Name is \(name)!"  
print(strinter)
```

When we execute above swift program in playground we will get result like as shown below

```
Hello My Name is Bharat Patel
```

Following is another example of string interpolation in swift programming.

```
let division = 1.0 / 4.0  
let strnumber = "The Division is \(division) as a decimal."  
print(strnumber)
```

When we execute above swift program we will get result like as shown below.

```
The Division is 0.25 as a decimal.
```

Swift Optionals

In swift, optional is a type which is used to hold the value or absence of value means nil values. Nil represent the absence of value.

Following is the simple example of defining variables without any initial value in swift programming language.

```
var firstname: String = "IOS Programming"  
var lastname: String // It will throw compile time error  
var uname: String = nil // It will throw compile time error
```

When we execute above variable statements we will get compile time error because we didn't assigned any value to "lastname" variable and we assigned "nil" to uname variable.

We got compile time error because in swift by default variables are non-optional. So if we define any variable without assigning any initial value or nil value we will get compile time problem.

To solve this problem and indicate absence of value the swift introduce type called “optional”. The optional will be defined by using question mark (“?”) operator after the type declaration.

Swift Optional Declaration

Following is the syntax of declaring optional value with variables.

```
var <VariableName>: Datatype<?>
```

If you observed above syntax we defined question mark (“?”) operator after the data type declaration.

Swift Optional Example

Following is the simple example of using optional in swift programming language to define optional type for variables.

```
var firstname: String?
var phonenumber: Int?
var character: Character?
print(firstname)
print(phonenumber)
print(character)
```

If you observe above example we declared three Optionals with different data types and we didn’t assigned any initial value for the variables.

When we execute above program in swift playground it won’t return any compilation errors instead we will get the result like as shown below.

```
nil
nil
nil
```

Swift Force Unwrapping

In swift if we define variable as optional means we need to unwrap the variable value using exclamation mark (!) otherwise we will get variable value with “Optional” keyword.

Following is the simple example of using Optional with variables to assign nil value in swift programming language.

```
var firstname: String?
firstname = "Bharat"
if firstname != nil {
    print("FirstName is \(firstname)")
}
else {
    print("No FirstName ")
}
```

If you observe above example we defined variable “firstname” with nil value so we will get result like as shown below.

FirstName is Optional("Bharat")

If you observe above result we got Optional keyword along with firstname variable value. To solve this problem we need to unwrap the variable value using exclamation mark (!).

Now we will add the exclamation sign (!) with the variables which we assign optional ("?").

```
var firstname: String?
firstname = "Bharat"
if firstname != nil {
    print("FirstName is \(firstname!)")
}
else {
    print("No FirstName ")
}
```

If you observe above example we added exclamation mark (!) to the optional variables in example.

Following is the result of above swift optional program with exclamation mark to unwrap the optional variable.

FirstName is Bharat

If you observe above result the Optional keyword removed from the string variable.

Swift Automatic Unwrapping

If you observe above example we are unwrapping optional value manually using exclamation mark (!). In case if you want to unwrap optional value automatically without doing any additional changes then we need declare variable with exclamation mark ("!") instead of question mark ("?").

Following is the example of unwrapping optional value automatically using exclamation mark ("!").

```
var name: String!
name = "Bharat Patel"
if name != nil{
    print("\(name)")
}
else {
    print("No Name ")
}
```

When we run above program we will get the result like as shown below.

Bharat Patel

Swift If Else Statement

Following is the syntax of if else statement in swift programming language.

```
If condition
{
    //Expressions will execute when condition TRUE
}
```

```
else
{
//Expressions will execute when if condition fails
}
```

If you observe above swift **if else** statement syntax first it will evaluates **if** condition if it **TRUE** then it will execute **if** condition expression otherwise it will execute **else** condition expressions.

Now we will see how to use if else statement in swift programming language with examples.

Swift If Else Statement Example

Following is the simple example using if else statement in swift to check the condition before execute the expressions within the condition.

```
var num1: Int = 10
if num1 > 20 {
print("Number Greater than 20")
}
else {
print("Number Less than 20 ")
}
```

If you observe above example we are checking the condition whether the value of “num1” greater 20 or not.

When we execute above code in swift playground we will get result like as shown below.

Number Less than 20

This is how we can use swift if else statement to execute expressions based on our defined conditions.

Swift Tuples

In swift, tuples are used to group multiple values together into single compound value. The values whatever we will define in tuple can be of any type and there is no restriction to use same type of values.

In swift we have two types of tuples are available those are unnamed tuples and named tuples.

Unnamed Tuples

In swift, unnamed tuples are used to group multiple values into single by using parentheses and those are need to be separated by commas.

Following is the simple example of defining unnamed tuples in swift programming language.

```
let userInfo = ("Bharat Patel", 200)
```

Or

```
var userinfo:(String, Int) = ("Bharat Patel", 200)
```

If you observe above unnamed tuple values we are grouping string and integer values into single compound value and we are using immutable `let` or mutable `var` based on our requirements.

Named Tuples

In swift, named tuples are used to group multiple values into single by using parentheses and need to define those values with named variables and those are separated by commas.

Following is the simple example of defining named tuples in swift programming language.

```
let userinfo = (name: "Bharat Patel", id: 200)
```

Or

```
var userinfo:(name:String, id:Int) = ("Bharat Patel", 200)
```

As we discussed in tuple we can use any type of values within the tuple and it's restricted to use same type like arrays.

If you observe above tuple example we defined values with variable names and we are grouping String and Integer type values into single tuple value.

In above examples we created immutable or unchangeable tuple using `let` and mutable tuple by using `var` keyword based on our requirement.

Now we will see how to access these tuples values in our swift programming language with examples.

Swift Access Tuple Values

Following is the simple example of accessing unnamed and named tuple values based on our requirements in swift programming language.

```
// Constructing a simple tuple
let tp1 = ("Bharat Patel", 200)
print(tp1.0)
print(tp1.1)
```

Swift For-in Loop

In swift, **for-in** loop is used to iterate over a sequence of items, such as items in an array list, characters in string or collection of items to execute specified statements repeatedly based on the defined conditions.

If we get requirement like need to execute set of statements repeatedly based on the items in list or array then by using **for-in** loop we can easily achieve this functionality in swift programming language.

Now we will see the functionality of **for-in** loop in swift programming language using algorithm diagram.

Syntax of Swift For-In Loop

Following is the syntax of defining **for-in** loop in swift programming language.

```
for item_name in list_name {  
    // your statements  
}
```

If you observe above swift **for-in** loop syntax we defined multiple parameters those are

Item_name – By using this we can access each item in defined list.

list_name – It's name of the list which we need to access the items.

Now we will see how to use **for-in** loop in swift programming language with examples.

Swift For-In Loop Example

Following is the simple example using for-in loop in swift to loop through the list of items.

```
var lstNames: [String] = ["Bharat","Khawar"]
```

```
for index in lstNames {  
    print( "Name of Developer is \(index)")  
}
```

If you observe above for-in loop example we are looping through the items in “lstNames” array list to print each item in list.

When we execute above program in swift playground we will get result like as shown below.

Name of Developer Bharat

Name of Developer Khawar

This is how we can use for-in loop in swift programming language to loop through the sequence of items in list based on our requirements.

Swift While Loop

In swift, while loop is used to execute set of statements continuously until, the defined condition is TRUE.

Generally in swift if we use for loop, we will clearly know that how many times the given statements will execute repeatedly but in while loop we don't know how many times the given statements will execute repeatedly until the defined condition fails (FALSE).

In swift while loop is useful whenever we need to execute given statements repeatedly till the defined condition is TRUE but not aware of how many times the statements will execute repeatedly.

In swift while loop will evaluates the condition before starting of the loop. Now we will see the functionality of while loop in swift programming language using algorithm diagram.

Syntax of Swift While Loop

Following is the syntax of defining while loop in swift programming language.

```
while condition {  
  // your statements  
}
```

If you observe above swift while loop syntax first it will check the condition and execute the statements till the defined condition fails.

Now we will see how to use **while loop** in swift programming language with examples.

Swift While Loop Example

Following is the simple example using while loop in swift to execute statements repeatedly till the condition TRUE.

```
var i = 1  
  
while i < 15 {  
  print(i)  
  i = i + 2  
}
```

If you observe above while loop example first it will check whether variable “i” value less than 15 or not. In case if it less than 15 then it will print the value and continue the loop till the defined condition fails (FALSE).

When we execute above program in swift playground we will get result like as shown below.

```
1  
3  
5  
7  
9  
11  
13
```

This is how we can use while loop in swift programming language to execute set of statements repeatedly till the defined condition fails based on our requirements.

Swift Repeat-While Loop

In swift, repeat-while loop is same as while loop but only the difference is, it will execute the defined statements first then it will perform the condition check and it will execute the statements repeatedly till the defined condition TRUE.

The repeat-while loop will execute the statements at least once even if the defined condition is fails (FALSE) because it will execute the statements first then it will perform condition check but in while statements execution will perform only when the defined condition is TRUE.

Note: The repeat-while loop in swift programming language is same as do...while loop in other programming languages.

Now we will see the functionality of repeat-while loop in swift programming language using algorithm diagram.

Syntax of Swift Repeat-While Loop

Following is the syntax of defining repeat-while loop in swift programming language.

```
repeat
{
// your statements
} while condition
```

If you observe above swift repeat-while loop syntax first it will execute the statements and then it will perform condition check.

Now we will see how to use **repeat-while** loop in swift programming language with examples.

Swift Repeat-While Loop Example

Following is the simple example using repeat-while loop in swift to execute statements repeatedly till the condition TRUE.

```
var sum = 2
```

```
repeat
{
print(sum)
sum = sum + 2
} while sum < 10
```

If you observe above repeat-while loop example first it will execute the defined statements and then it will perform condition check and execute statements continuously till the defined condition is TRUE.

When we execute above program in swift playground we will get result like as shown below.

```
2
4
8
10
```

This is how we can use repeat-while loop in swift programming language to execute statements at least once even if defined condition is FALSE based on our requirements.

Swift Collection Types (Arrays, Sets, Dictionaries)

Swift Arrays

In swift, **arrays** are used to store same data type of multiple values in ordered manner. In **array** we can store same value in multiple times at different positions based on our requirements.

In swift arrays we can store any kind of elements like either strings or integers or other element types based on our requirements.

We can use arrays in swift programming language whenever we want to hold multiple values of single data type in one variable.

Following is the pictorial representation of **arrays** in swift programming language.

By using index we can access arrays elements and perform update or delete operations on array elements based on our requirements in swift programming language.

In swift we can create arrays by surround a list of comma separated values with square brackets.

Arrays Declaration

Following are the different ways of defining or declaring arrays in swift programming language.

Simple way of defining an array elements with comma separated values surrounded by square brackets.

```
// An array of integer elements  
var numbers = [1, 2, 3, 4]
```

Following is another way of create an array with data type in an array declaration.

```
var nums: Array<Int> = [1,2,3]  
print(nums)  
// It prints [1, 2, 3]
```

Following are the different ways of create an **empty arrays** with data type in an array declaration.

```
var intvals = Array<Int>()  
var intarr = [Int]()
```

Accessing Array Elements

In swift we can access array elements by using **index** positions. Generally in arrays **index** position will start from **zero (0)**.

Check Array Empty or Not

By using **isEmpty** property we can check whether the given array contains elements or not in swift programming language.

Get Array Elements Count

In swift by using **count** property we can get the **count** or **number** of elements exists in an array.

Add Elements to an Array

In swift we can add elements to an array by using built-in **append()** method or **+=** operator based on our requirements.

Suppose if we want to insert elements at particular index position of an array then by using **insert()** method we can insert elements in an array based on our requirements.

Remove Array Elements

In Swift, we can easily delete or remove the elements from an array by using **removeLast()** and **removeAtIndex()** methods based on our requirements.

Now we will see how to delete array elements based on index position by using **removeAtIndex** method.

Update an Array Elements

In swift we can easily update an array elements by assigning new values based on index positions.

Iterating Over an Array Elements

By using for-in loop we can iterate or loop through an elements in array based on our requirements in swift programming language.

Swift Dictionaries

In swift, **dictionaries** are used to store values with key-value pair association in unordered manner. In dictionaries we can store same type of **keys** and same type of **values** with no defined order. The **keys** in dictionaries must be unique and those will act as identifiers for the **values** in dictionaries.

We can use dictionaries in swift programming language whenever we want to get values based on **key** value.

In swift we can create **dictionaries** by using **dictionary** literal. The **dictionary** literal contains a list of comma separated key-value pairs, in which a colon separates each key from its associated value surrounded by square brackets.

We can assign dictionary literal to any variable or constant or any function in swift programming language based on our requirements.

Syntax of Swift Dictionaries

Following is the syntax of defining dictionary collection types in swift programming language.

```
Dictionary<Key, Value>()
```

Or

```
[Key: Value]()
```

In above syntax the **Key** is the type of value that can be used as a dictionary key and **Value** is the type of value that dictionary stores for the defined keys.

Creating a Dictionaries in Swift

Following are the different ways of creating an **empty dictionaries** with **key-value** pair type in declaration.

```
// using Dictionary
let nums = Dictionary<Int, String>()
print(nums.count)
// It prints 0

// without dictionary
let nums = [Int: String]()
print(nums.count)
// It prints 0
```

The above examples will create an empty dictionary of [**Int**, **String**] to store **Int** type of keys and **String** type of values.

Following are the different ways of defining or declaring dictionaries with keys and values in swift programming language.

```
// with Dictionary Literal
var names = Dictionary<String, Int>()
names = ["Bharat": 1, "Rohini": 2]
print(names)
// It prints [Bharat: 1, Rohini: 2]

// without Dictionary Literal
var names = [Int: String]()
names = [1: "Bharat", 2: "Rohini"]
print(names)
// It prints [1: Bharat, 2: Rohini]
```

Accessing Dictionary Elements in Swift

In swift we can access dictionary elements by using key values based on our requirements.

Following is the example of accessing elements in **Dictionary** using key values.

```
// with Key-Value Data Types
var names = [Int: String]()
names = [1: "Bharat", 2: "Rohini"]
print(names)
print(Array(names.keys))
print(Array(names.values))
print(names[1])
print(names[2])
```

If you observe above example we defined dictionary object “**names**”, Key-Value pair **data type** and accessing dictionary values in different ways.

When we run above program in swift playground that will return result like as shown below

```
[1: Bharat, 2: Rohini]
[1, 2]
[Bharat, Rohini]
Bharat
Rohini
```

Check Dictionary Empty or Not in Swift

By using **isEmpty** property we can check whether the given dictionary contains elements or not in swift programming language.

Get Dictionary Elements Count in Swift

In swift by using **count** property we can get the count or number of elements exists in dictionary.

Add Elements to Dictionary in Swift

In swift we can insert or add new item to a dictionary object by using new key of appropriate type and assign new value of appropriate type.

Update or Modify Elements in Dictionary

In swift we can modify or update dictionary object element values by assigning new values to appropriate keys.

We have another alternative way, **updateValue(, forKey:)** method in swift programming language to set or update the values of particular key in dictionary object.

Remove Dictionary Elements in Swift

In Swift, we can easily delete or remove the key-value pair in dictionary by assigning a value of **nil** to the respective key and alternatively we can use **removeValueForKey()** method to remove key-value pair.

There is also another way of removing elements in dictionary by using **removeValueForKey()** method.

Iterating Over a Dictionary in Swift

By using for-in loop we can iterate or loop through an elements in dictionary objects and use it in our applications based on our requirements in swift programming language.

Swift Sets

In swift, **sets** are used to store same type of distinct values in unordered manner. We can use **sets** instead of arrays in case if we want to store only distinct values and we are not concerned about the order of items.

Following is the pictorial representation of **sets** in swift programming language.

In swift we can create **sets** same arrays like surround a list of comma separated values with square brackets except by specifying Set type.

Syntax of Swift Sets

Following is the syntax of defining set collection types in swift programming language.

```
Set<ElementType>
```

Here **ElementType** is used to specify data type of values which we are going to store in sets.

Create a Sets in Swift

Following are the different ways of defining or declaring sets in swift programming language.

One way of defining sets, elements with comma separated values surrounded by square brackets along with **Set** type specified.

```
// Set of integer elements
var numbers: Set = [1, 2, 3, 4]

// Set of string elements
var names: Set = ["Bharat", "Rohini", "Praveen", "Sateesh"]
```

Following are the different ways to create sets with data type in set declaration.

```
var nums: Set<Int> = [1,2,3]
print(nums)
// It prints [1, 2, 3]

var names: Set<String> = ["Bharat", "Rohini", "Praveen", "Sateesh"]
print(names)
// It prints [Bharat, Rohini, Praveen, Sateesh]
```

Following is the one way of creating an **empty sets** with data type in **sets** declaration.

```
let nums = Set<Int>()
print(nums.count)
// It prints 0
```

Accessing Set Elements

In swift we can access Set elements by using some of built-in methods like contains, first, etc.

Following is the example of checking whether Set contains particular element or not using **contains()** method and get first element in **Set** using **first** property.

```
var names = ["Bharat", "rohini", "praveen", "sateesh"]
if names.contains("Bharat") {
    print("Set Contains Element")
} else {
    print("Element Not Exists")
}
```

Check Set Empty or Not

By using **isEmpty** property we can check whether the given set contains elements or not in swift programming language.

Get Set Elements Count

In swift by using **count** property we can get the count or number of elements exists in sets.

Add Elements to a Set

In swift we can insert or add elements to a set by using **insert()** method.

Remove Set Elements

In Swift, we can easily delete or remove the elements from Set by using **remove()** method.

Iterating Over a Set Elements

By using for-in loop we can iterate or loop through an elements in sets based on our requirements in swift programming language.

In swift, **Set** type elements doesn't have any defined order. We can iterate over **Set** of elements in a specified order by using **sorted()** method.

Swift Set Operations

In swift we can perform some of mathematical operations like Union, Intersection, Subtract, etc. on **Sets** based on our requirements.

Suppose if we have two Sets, we easily perform mathematical operation like Union, Intersection and etc. based on our requirements.

Union Operator

In swift union operator is used to create new Set by combining the values of two sets.

Intersect Operator

In swift Intersect operator is used combine multiple Sets and create new Set with the common elements of multiple sets.

Subtract Operator

In swift subtract operator is used to remove the values which are same in both the Sets and return remaining elements in first Set.

Swift Functions

In swift, **functions** are the block of statements to perform specified tasks and return values based on our defined conditions.

In swift we can send data to functions by using input parameters and functions is having return types to return data as result.

Generally in swift, functions will help us to improve the code re-usability like suppose if we have requirement like implementing the same functionality in multiple places of an application then we can create single function with the required code in separate file and call that function in our application wherever it required and by doing this automatically the duplicate code in our application will get reduced.

Syntax of Swift Functions

In swift the declaration of function syntax will be same as other programming languages but only difference is **return type** must be declare in parameters side.

In swift functions the declaration of parameters and return type are not a mandatory and by using “**func**” keyword we can define functions.

Following is the syntax of defining or declaring functions in swift programming language.

```
func functionName(parameters) -> Returntype {  
  //Statements  
  return parameters  
}
```

If you observe above swift functions syntax we defined few parameters those are

func – It’s a keyword which is used to define function in swift programming language.

functionName – Its name of the function which we are going to create in application.

parameters – These are the input parameters to pass data to functions and its optional.

-> - The hyphen followed by a right angle bracket indicates return type of function in swift.

Returntype – It indicates return type of the function.

Swift Function with Parameters and Return Types

In swift, functions will accept input parameters and return values based on our requirements. The input parameters and return values are optional in swift functions.

Now we will see how to use parameters and return types in swift programming language based on our requirements with examples.

Swift Function without Parameters

Generally in swift functions it's not a mandatory to define input parameters. Following is the example of defining function without having any parameters in swift programming language.

```
func show() {  
    print("Welcome to IOS Programming")  
}  
show()
```

If you observe above example we defined function without having any input parameters and return types in swift programming language.

When we run above program in swift playground we will get result like as shown below

```
Welcome to IOS Programming
```

Now we will see how to use functions with multiple parameters in swift programming language.

Swift Function with Multiple Parameters

In swift functions we can send multiple input parameters based on our requirements.

Following is the example of sending single parameter to the function in swift programming language.

```
func addition(a: Int) {  
    print("\(a) + 50 = \(a + 50)")  
}  
addition(120)
```

In above example we are sending only one parameter to the function in swift programming language.

When we run above program in swift playground we will get result like as shown below

```
120 + 50 = 170
```

Swift Classes

In swift, **classes** are the building blocks of our programming code in which we declare our methods, properties, variables and etc. to extend the functionality of classes.

In swift we can define classes using class keyword and define the properties, methods in classes same as constants, variables and functions.

Following are the advantages of using classes in swift programming language.

- We can implement inheritance to acquire the properties from one class to another.
- Type Casting enable us to check the type of class at runtime.
- Deinitializers in classes will take care of memory management by releasing unused resources.
- We can create more than one reference for class instance.

In swift, classes are reference to the same existing instance when they are passed around in our code. Suppose if we assign same class to multiple variables and if we modify one, all the variables with same class reference will be updated with new values because classes are **reference** types.

Unlike other programming languages we don't need to create any separate interface or implementation files for classes. In swift when we create a classes in single file, automatically that class file will be available for other code blocks to use that functionality in applications.

Syntax of Swift Classes

Generally, in swift classes are defined by using class keyword. Following is the syntax of defining classes in swift programming language.

```
class ClassName {
// Variables
// Methods
// Properties
}
```

If you observe above syntax we defined class using class keyword along with class name and we defined a required properties, variables and functions in structures based on our requirements in swift programming language.

Swift Class Instances

Generally, in swift we can access class properties, variables and methods by creating class instance like as shown below.

Following is the example of creating instance for newly created class in swift programming language.

```
class Room {
var TeacherName : String
var StudentName : String
var MonitorName : String
}
var sd = Room()
```

If you observe above example we created instance for **Room** by using that we can access the properties of class based on our requirements.

Key Points of Classes

Following are the some of key points which we need to remember while using **classes** in our swift programming language.

- Classes are the pointers towards the “Objects”
- Classes allow you to declare properties, methods and subscript.
- Classes allow to inherit protocols.
- Classes must have initializer and DE initializer.
- Classes can inherit from another class.

This is how we can use classes in swift programming language based on our requirements.

Swift Inheritance & Overriding

In swift, **inheritance** means acquiring the **properties** from parent class to child class. By using inheritance, a class can inherit properties, methods and other characteristic from another class.

In inheritance, the class which is inheriting properties, methods from another class is called a **subclass** and the class which contains properties, methods and functions to inherit other classes is called a **superclass**.

The classes in swift can inherit the properties, methods and functions from super class using inheritance and can implement custom functionality in sub class by overriding the properties and methods of super class based on our requirements. Swift will ensure overrides are correct by checking that the override definition has a matching super class definition.

In classes we can add property observers to inherited properties to notify about the changes made for the properties.

Swift Base Class

In swift, a class that doesn't inherit properties, methods or functions from any other class is called as a **Base Class**.

Now we will see how to define base class in swift programming language with examples.

Following is the example of defining base class with properties and methods in swift programming language.

```
class person {
var Name: String = ""
var Age: Int = 0
var Gender: String = ""
init(Name: String, Age: Int, Gender: String)
{
self.Name=Name
self.Gender=Gender
self.Age=Age
}
}
let p = person.init(Name: "Bharat Patel", Age: 30, Gender: "Male")
```

```
print(p.Name,p.Age,p.Gender)
```

If you observe above example we defined a base class “**person**” with properties like **Name**, **Age**, **Gender** and **init** method to initialize property values.

When we run above example in swift playground we will get result like as shown below

```
Bharat Patel 30 Male
```

This is how we can define a base class in swift programming language based on our requirements.

Swift Sub Class

In swift, a class which is inheriting properties or methods from other existing class is called as a **subclass**. In subclass we can override the properties or methods of base or existing class with same definition based on our requirements.

In swift we can define subclass by using **colon** (“:”) before the base class name. Following is the syntax of defining subclass in swift programming language.

```
class subclass: superclass {  
    // subclass implementation here  
}
```

In above syntax we defined subclass name before the **superclass** and both are separated by **colon** (“:”). In **subclass** we can inherit the properties or methods from superclass based on our requirements.

Swift Overriding

In swift, Overriding is the process in which subclass is fully responsible to change or re-implement the instance method, instance property and type property which is define in parent class or superclass.

A subclass provides fully custom implementation for any type including instance method or property of super class based on our requirements.

In swift to override inherited properties or methods in subclass we use override keyword which will tells the compiler to check the overriding method or property definition matches with the base class or not.

Syntax of Swift Overriding

Following is the syntax of overriding inherited properties or methods in subclass using override keyword.

```
class Sample1 {  
    func Name() {  
        // Function implementation here  
    }  
}  
  
class Sample2: Sample1 {  
    override func Name() {
```

```
// Override function implementation here
}
}
```

If you observe above syntax we defined base class “**Sample1**” and inheriting base class properties and methods in subclass “**Sample2**” and overriding function in subclass using override keyword.

Swift Protocols

In swift, **Protocols** are used to define a skeleton of classes, structures, methods, types and that would require other types to implement a defined classes, methods, properties. The defined protocol adopted by other classes or methods to provide an actual implementation of defined properties or methods.

If we work on other programming languages the term **protocol** is similar to **interfaces** in Java and C#. Protocols only responsible for the signature of methods and functions.

Generally, in swift we use a protocol keyword to define a protocols in our applications.

Swift Protocol Syntax

The syntax of protocol is very similar to class, structure and enumeration but there is only a difference is protocol keyword and implementation type section.

Following is the syntax of defining a protocol in swift programming language.

```
protocol ProtocolName {
// Structure Body Here
}
```

We can adopt the defined protocol in structure or a class by placing a protocol name after the type name which is separated by **colon** (:) during its declaration. In case if we have multiple protocols to adopt those are separated by **commas** (,) like as shown below

```
struct fullnameStructure: FirstNameProtocol, LastNameProtocol {
// Structure Body Here
}
```

In case class have a superclass, we need to list the class name first then we need to add the protocol names which we are going to adopt like as shown below

```
class fullnameClass: ParentClass, FirstNameProtocol, LastNameProtocol {
// Class Body Here
}
```

Swift Protocol Inheritance

In Swift, **Protocols** can inherit one or more protocols and further modify with its on requirement. The syntax of protocol inheritance is similar to class inheritance, but to list a multiple inherited protocols we need to separate them with commas.

Swift Protocol Extensions

Swift allow programmers to extend their protocols to provide method and property implementation. This feature allows programmers to define protocols behaviors themselves rather than implement at individual way in a global function.

Swift Extensions

In swift, **Extension** is the process of extending the functionality of an existing classes, structures, enumerations or protocol types by adding a new functionalities.

By using extensions we can extend the functionality of types which we don't have access to the original source code. In swift, extensions can provide the following functionalities.

- It can add a computed type instance properties
- It can define an instance and type methods
- Define a new nested types and subscripts

Generally, in swift we use extension keyword just before the class, structure or enumeration to extend the functionality based on our requirements.

Swift Extension Syntax

Following is the syntax of defining an extensions using extension keyword in swift programming language.

```
extension NameofType {  
//New functionality and types  
}
```

Extensions also extend to adopt one or more protocols. After declare of extension type we can inherit protocols using **semicolon** (:) sign and multiple protocols can be separated by **commas** (,).

```
extension NameofType: Protocol1, Protocol2 {  
//implementation of protocol  
}
```

Swift Extend Computed Properties

Computed properties are those properties which are calculated each and every time in our code but can't store the value. These type of properties are used in classes, structures and enumerations. Extensions allow user to add computed properties and computed type properties in the class.

Swift Initialization

In swift, **initialization** is a special type of method in class, structure or enumeration and it is responsible for making sure a newly created instance of class, structure or enumeration is fully initialized and perform other tasks that is required before the new instance is ready for use in our applications.

In swift we can implement **initialization** process by defining special methods called **initializers**, these methods will call during the creation of new instance of particular type and these initializers are responsible for making sure that new instances of particular type correctly initialized before they are used for the first time.

Generally in classes, structures and enumerations we must need to set the initial values for all the properties which are defined by the time of an instance created for the class or structure.

In swift, we can set the initial values for class, structure stored properties within the initializer or we can set the default values for defined properties in class or structure as a part of the definition.

In swift initializers are represented by `init` keyword and these initializers are called during the creation of new instance for particular class or structure.

Swift Initializer Syntax

Following is the syntax of defining initializer in swift programming language.

```
init() {  
    // initialization here  
}
```

If you observe above syntax we defined a initializer using `init` keyword and the initialization of class or structure properties can be done within the initializer **`init()`** method.

Swift Initialization with Parameters

In swift initializer **`init()`** method we can send initialization parameters as a part of its definition same like functions and methods parameters.

Swift Initialization with Multiple Initializers

We can use multiple initializer **`init()`** methods to initialize properties based on our requirements.

Swift Initializer with Parameter Names and Argument Labels

In swift, initialization parameters can have parameter names and argument labels. In initialization parameter names are used within the initializer body and argument labels are used when calling the initializer.

In swift, initializers doesn't have any identifying name like functions or methods so the initialization parameters will play an important role to identify which initializer should be called and swift provides an automatic argument labels for every parameter in an initializer **`init()`** method in case if we didn't provided one.

Following is the example of defining initializer **`init()`** method with or without argument labels and parameter names.

```
class concatenate {
```

```

let fname, lname : String

init(fname: String, lname: String ) {
self.fname = fname
self.lname = lname
}
init(firstname: String, lastname: String ) {
fname = firstname
lname = lastname
}
}
var name = concatenate(fname: "Bharat", lname: "Patel")
print("Full Name: \(name.fname) \(name.lname)")
var uname = concatenate(firstname: "Rohini", lastname: "Alavala")
print("Full Name: \(uname.fname) \(uname.lname)")

```

If you observe above example we created an initializers only with parameter names so we used parameter names to call the initializers.

When we run above example in swift playground we will get result like as shown below

```

Full Name: Bharat Patel
Full Name: Rohini Alavala

```

Note: In case if we define argument labels in initializers it's mandatory to call the initializer with argument labels otherwise we will get compile time error.

Swift Default initializers

Swift provides a default initializer for class or structure to set default values for all the properties in class or structure. The default initializer will create a new instance for class or structure with all of its properties to set their default values.

Following is the example of defining a class **Student**, with three properties **name**, **position** and **location** in swift programming language.

```

class Student {
var name: String?
var position = "Team Lead"
var location = "Hyderabad"
}
var stu = Student()
print(stu.name)
print(stu.position)
print(stu.location)

```

In above example we created a class **Student** with three properties and assigned default values. The **Student** class will behave as a default initializer and that will creates a new instance with all of its properties to set their default values.

When we run above example in swift playground we will get result like as shown below

```

nil
Team Lead

```

This is how we can use default initializers in swift programming language based on our requirements.

Swift Structures

In swift, **structures** are used to construct building blocks of our programming code and we can extend the functionality of structures by defining different type of properties and methods.

In swift we can define structures using struct keyword and define the properties, methods in structures same as constants, variables and functions.

In swift structures are quite similar as classes to group similar information together based on our requirements. One of the main difference is structures are **value types** and classes are **reference types**.

Following are the common things in both structures and classes in swift programming language.

- Defining properties to hold values
- Defining methods to implement required functionality
- Defining initializers
- Defining type methods
- Defining subscripts, Protocols, Extensions

In swift, structures are always copied when they are passed around in our code because when we assign same structure to multiple variables and if we modify one, only that value alone will change, not others because each new variable was a copy of the original, not a reference.

Unlike other programming languages we don't need to create any separate interface or implementation files for structures. In swift when we create a structures in single file, automatically that class file will be available for other code blocks to use that functionality in applications.

Syntax of Swift Structures

Generally, in swift structures are defined by using struct keyword. Following is the syntax of defining structures in swift programming language.

```
struct StructName {  
    // Variables  
    // Methods  
    // Properties  
}
```

If you observe above syntax we defined structure by using struct keyword along with structure name and we defined a required properties, variables and functions in structures based on our requirements in swift programming language.

Swift Structure Instances

Swift Assign and Access Structure Properties

Swift Functions in Structures

In swift structures we can define functions along with properties based on our requirements.

Key Points of Structure

Following are the some of important points which we need to remember while using **structures** in our swift programming language.

- Structures are the value type when we pass its value to variable the original value of variable will be remain same.
- By using Dot Syntax, we can access all the name types (properties, methods) which we declared in structure.
- Structures have default initializer but we also set the custom initializer.
- At initialization level, it's mandatory to initialize defined properties with values.

Difference between Classes and Structures

In swift, classes have some of additional capabilities when compared with structures. Following are the differences between classes and structures in swift programming language.

- No inheritance in Structures. It's not possible for us to Inherit from other types.
- Classes use **type-casting** in swift to define a class as a superclass or subclass but **type-casting** not available for structures.
- By default no mutation available for structure methods. In case if we try to change the value of property, compiler will throw an error.
- Deinitializers not available in structures to free up assigned resources in instance.
- Not possible for us to create multiple references to the same instance in Structures.

Choose Between Classes and Structures

In swift both structures and classes is having their own priorities so we need to consider following things to decide whether to use class or structure in swift programming language.

- In swift if we want to share the mutable state between different parts of application then we need use a class.
- Structures are significantly faster, use less memory and useful for few simple data values.
- In structures **inheritance** will not support so if we want implement inheritance functionality we need to go for classes.
- The properties defined in structures are value types so the values will be copied rather than referenced when we pass around an instance of the structure.
- In structures we don't need to inherit properties from another existing type.

By considering above mentioned points you can decide which one is better to use in your swift applications based on your requirements.

Swift Enumerations

In swift, **enumeration** is a data type which consists a set of related values and it allow us to use those defined values in our code in a type-safe manner.

The values whatever we defined in swift enumeration can be a string or a character or a value of integer or float. We can create different type of enumeration examples in swift like collection of colors, cards, animals and etc.

The swift enumeration is powerful when we compare with other programming languages such as Objective-C and etc. because enumeration share their functionalities with structures and class. An enumeration in swift is a user defined so it contains methods, protocol and properties.

Swift Enumeration Syntax

Generally in swift we use `enum` keyword to define a new enumeration followed by its name and the values in enumeration can be defined with case statement.

Following is the syntax of using enumeration in swift programming language.

```
enum enumname{  
//Case A  
//Case B  
//Case C  
-----  
//Case N  
}
```

If you observe above swift enumeration syntax we used **enum** keyword followed by its name and the values in enumeration defined using case statement.

Now we will see how to use enumerations in swift programming language with examples.

Super and Self keyword

We can access super class members using the `super` keyword in Swift.

To access a super class variable, we can use `super.(variablename)`

To access a super class method, we can use `super.(methodname())`

To access a super class constructor, we can use convenience initializer in subclass constructor

To access class method within another method in the same class without using object, we can use `self.(methodname())`.

To override and access constructor variables with same name as argument within constructor, we can use `self.(variablename)`.